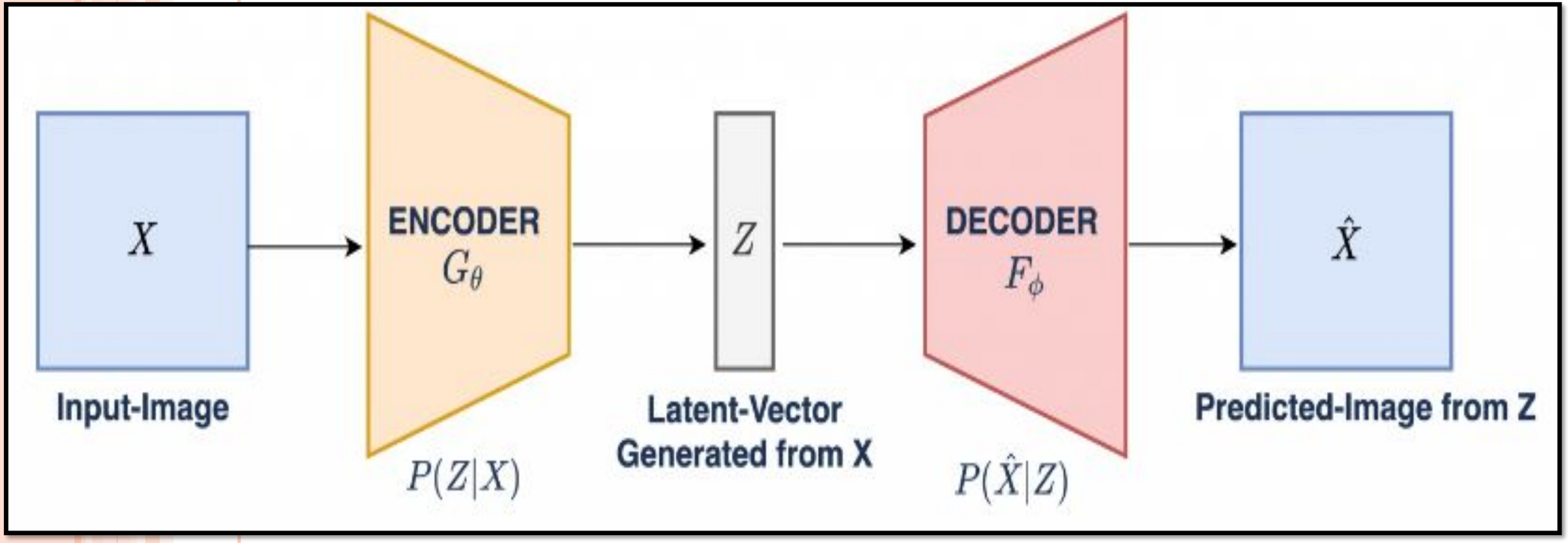
UNIT-II

Auto Encoders



CONTENTS

- Introduction
- Architecture of Autoencoder
- Mathematical representation of Autoencoder
- Example of Autoencoder
- Properties of Autoencoder
- Hyper parameters of Autoencoder
- Training of Autoencoder
- Applications of Autoencoder
- Use case and Limitations of Autoencoders

INTRODUCTION

- **Autoencoders** fall under **unsupervised learning algorithms** as they learn the **compressed representation of the data automatically** from the **input data without labels.**
- **In Autoencoders Name,**
  - **AUTO means =self**
  - **ENCODERS means- Convert into a different form (reduced Dimensions)**
- **Autoencoders (AEs)** are a type of **neural network architecture** that is able to find a ***compressed representation*** of the **input data** such as **image, video, text, speech, etc..**
- They became a popular solution for **reducing noisy data.**
- **Autoencoders** are used to **reduce the size of our inputs** into a **smaller representation**. If anyone **needs the original data**, they can **reconstruct it from the compressed data**.
- **Autoencoders** are a **specific type of feed forward neural networks trained to copy** its **input to output.**

INTRODUCTION

The **aim of an autoencoder** is to **learn a lower-dimensional representation (encoding)** for a **higher-dimensional data**, typically for **dimensionality reduction**, by **training the network to capture the most important parts of the input image.**

ARCHITECTURE

01 ENCODER

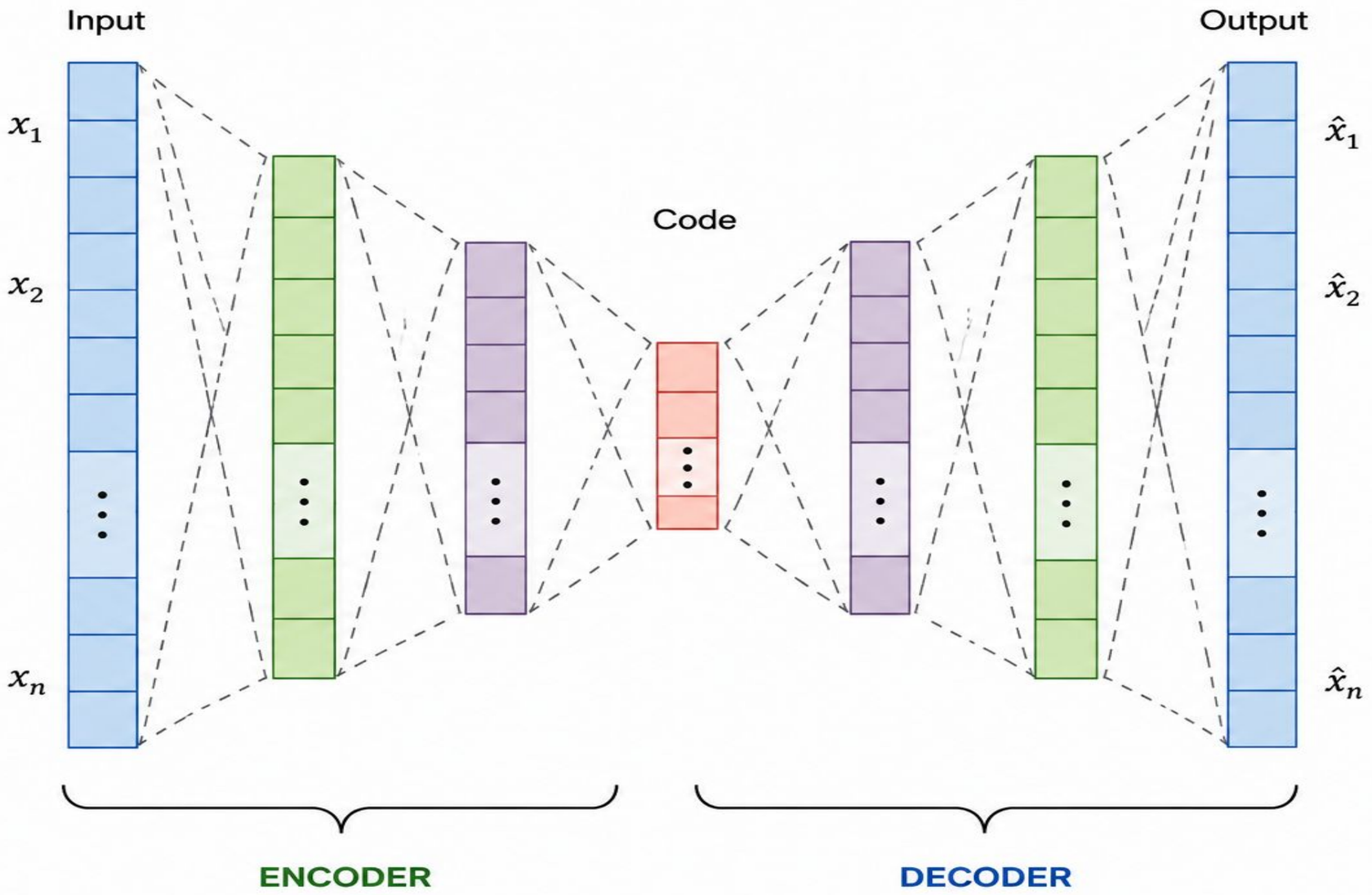
Maps the input data to a lower-dimensional representation.

02 CODE

Compressed latent representation of the input.

03 DECODER

Reconstructs the input data from the latent representation.



- Autoencoders consist of **3 parts:** **1. Encoder** **2. Code** **3. Decoder**
- An **Autoencoder replicates the input data to the output** in an **unsupervised manner** and is therefore sometimes referred to as a **replicator neural network.**

1. **Encoder:** An encoder is a **feed forward, fully connected neural network.**

The **encoder** is the **first part of the autoencoder** and is responsible for t**ransforming the input data into a compressed representation**. This compressed representation is often referred to as the **latent space or bottleneck or Code**

The encoder consists of **one or more layers of neurons,** typically implementing **non-linear activation functions like ReLU (Rectified Linear Unit)** to capture complex patterns in the input data.

It is mathematically represented as

$$h=f(x)$$

here

x = Input data

f(·)= Encoder function (a neural network)

h = Hidden representation (Latent vector/Code)

The encoder learns to keep only the **most important information** from the input while removing unnecessary details.

Example: Suppose the **input image** contains **784 pixels** (28 × 28 MNIST image).

Input : $x=[x1,x2,\ldots,x784]$

After passing through the encoder,

$h=[h1,h2,\ldots,h32]$

Now,

Original information = 784 values

Compressed information = 32 values

These 32 values summarize the important features of the image.

2. Code: Code is the representation of the **compressed input** that is applied **to the decoder**. This part of the network is also referred to as the **bottleneck**. It **balances two factors** such as **which part of the information to take** and **which part of the information to discard.**

The layer between the **encoder and decoder**, ie. the code is also known as **Bottleneck**. The bottleneck layer is the layer in the **middle of the autoencoder**, where the **compressed representation is stored**.

It has a **smaller number of neurons compared to the input and output layers,** effectively **reducing the dimensionality of the data.**

3. Decoder: This layer decodes the **encoded image back to the original dimension.** The decoded image is a **lossy reconstruction of the original image** and it is **reconstructed from the latent space representation.**

The decoder r**econstructs the original input** from **the compressed representation**.

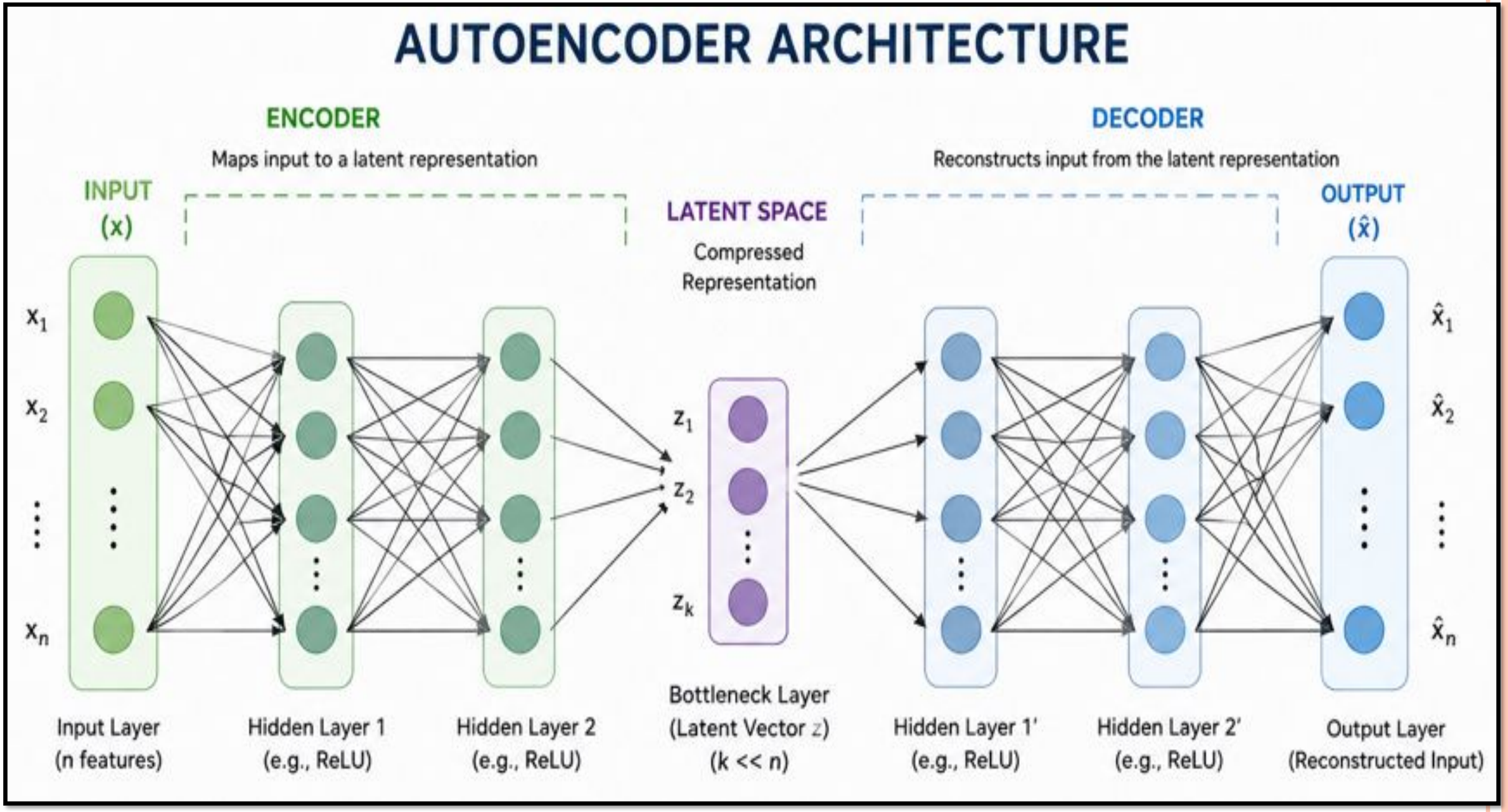
It is represented as **x^=g(h)**

where

g(·) = Decoder
h = Latent representation
x^ = Reconstructed output

Substituting **h=f(x)**

Gives **x^ =g(f(x))**


AUTOENCODER ARCHITECTURE
ENCODER
Maps input to a latent representation
DECODER
Reconstructs input from the latent representation
INPUT
(x)
LATENT SPACE
Compressed
Representation
OUTPUT
(x̂)
x1
x2
xn
z1
z2
zk
x̂1
x̂2
x̂n
Input Layer
(n features)
Hidden Layer 1
(e.g., ReLU)
Hidden Layer 2
(e.g., ReLU)
Bottleneck Layer
(Latent Vector z)
(k << n)
Hidden Layer 1'
(e.g., ReLU)
Hidden Layer 2'
(e.g., ReLU)
Output Layer
(Reconstructed Input)

Overall Autoencoder :

The complete autoencoder is

$$\hat{x} = g(f(x))$$

Where,

Encoder compresses the input.

The decoder reconstructs the compressed information.

Goal of Autoencoder :

The Goal is : $\hat{x} \approx x$

which means,

The **reconstructed output** should be **as close as possible to the original input.**

The **autoencoder is trained** so that the **difference between x and x^ becomes very small.**

This difference is called the **reconstruction error**.

Q: Why Can't the Output Be Exactly Equal to the Input?

Ans: Suppose you have a **100-page book**.

- **Your friend says**, "Read the **entire book** and explain it to me in just **5 pages**.“

  Can you write every single sentence from the book? **NO**

  You can only write the **important points**.

  Some **small details will be lost.**

- Now **imagine your friend tries to rewrite the original 100-page book using your 5-page summary.**

- Will it be **exactly** the same as the original book?

  No.

  It will be **very similar**, **but not identical.** This is exactly what happens in **an autoencoder.**

Mathematical Representation

Essentially, we split the **network into two segments**, the **encoder, and the decoder**.

$$\phi : \mathcal{X} \rightarrow \mathcal{F}$$
$$\psi : \mathcal{F} \rightarrow \mathcal{X}$$
$$\phi, \psi = \underset{\phi,\psi}{\arg\min} \, ||X - (\psi \circ \phi)X||^2$$

- The **encoder function**, denoted **by ϕ,** maps the **original data X,** to a **latent space F,** which is present at the **bottleneck.**
- The **decoder function**, **denoted by ψ**, maps the **latent space F** at the **bottleneck to the output.**
- The **output,** in this case, is the **same as the input function**. Thus, we are basically **trying to recreate the original image** after some

- The **encoding network** can be represented by the **standard neural network function passed through an activation function**, where **z is the latent dimension**

$$\mathbf{z} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- Similarly, the **decoding network** can be **represented in the same fashion,** but with **different weight, bias,** and **potentially activation functions being used.**

$$\mathbf{x}' = \sigma'(\mathbf{W}'\mathbf{z} + \mathbf{b}')$$

Mathematical Representation

- The **loss function** can then be written in terms of **these network functions,** and it is this **loss function that we will use to train the neural network** through the standard **back propagation procedure.**

$$\mathcal{L}(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|^2 = \|\mathbf{x} - \sigma'(\mathbf{W}'(\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})) + \mathbf{b}')\|^2$$

Properties of Autoencoders

1. Unsupervised Leaning: Autoenoders are considered as **unsupervised learning technique**. Since they **don't need explicit labels to train on.**

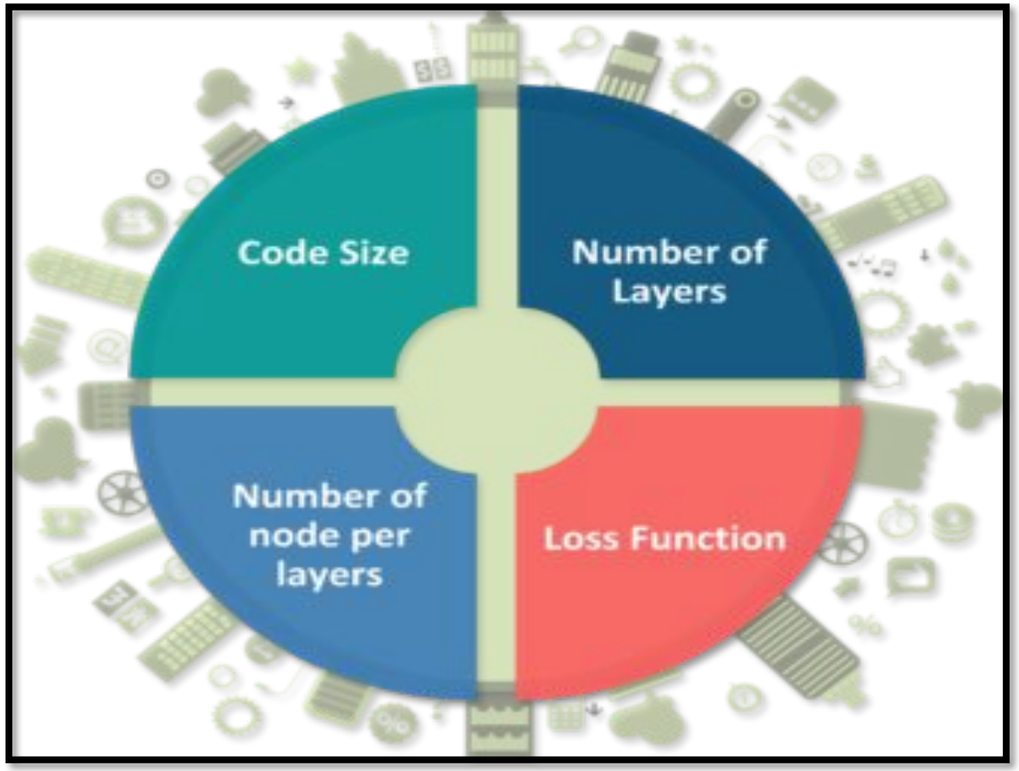
2. Data-specific: Autoencoders are only able to **compress data** similar to **what they have been trained on**.

3. Lossy: The **decompressed outputs** will **be degraded compared** to the **original inputs.**

4. Learned automatically from examples: It is **easy to train specialized instances of the algorithm** that will perform well on a **specific type of input.**

Autoencoders

Hereare **4 hyperparameters** that we need to set before **training an autoencoder:**



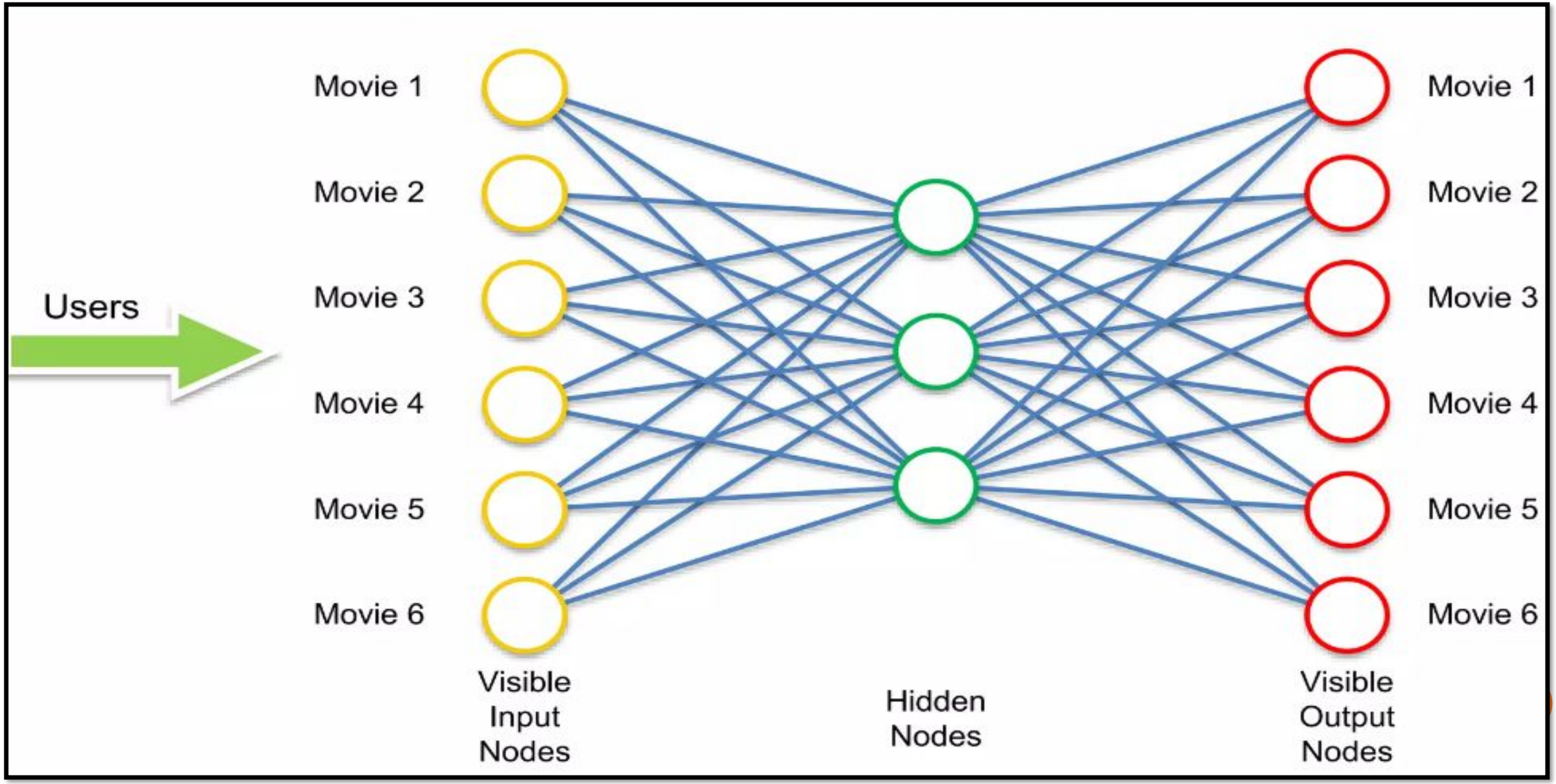
Code size: It represents the **number of nodes in the middle layer.** Smaller size results in **more compression.**

Number of layers: The autoencoder can consist of as **many layers as we want.**

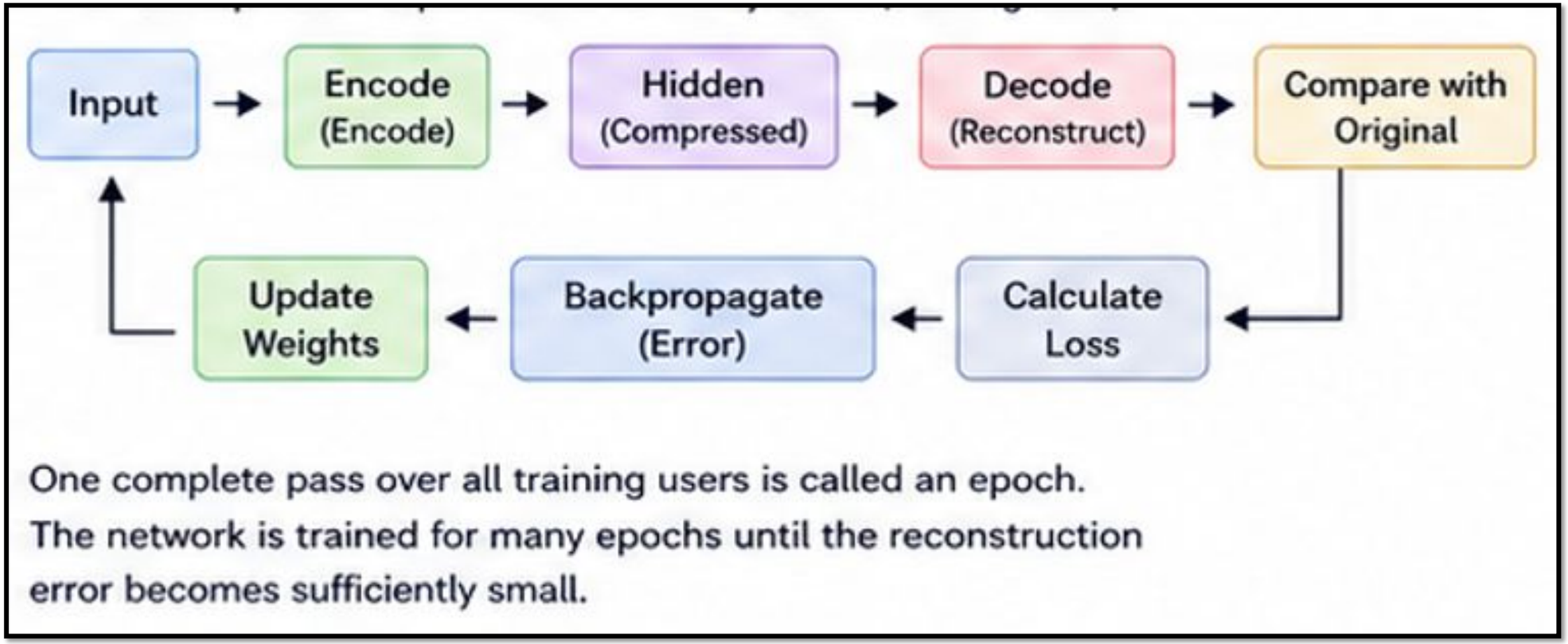
Number of nodes per layer: The number of nodes per layer **decreases with each subsequent layer of the encoder**, and **increases back in the decoder.** The **decoder is symmetric to the encoder** in terms of the **layer structure.**

Loss function: We either use **mean squared error** or **binary cross-entropy.** If the input values are in the **range [0, 1]** then we typically use **cross-entropy**; otherwise, we use the **mean squared error.**

Training of Autoencoders



Training of Autoencoders

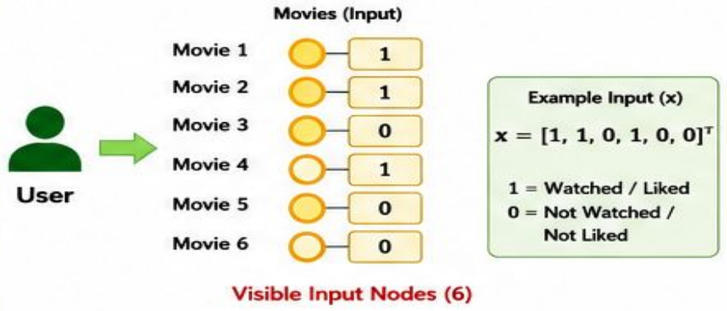


One complete pass over all training users is called an epoch.
The network is trained for many epochs until the reconstruction error becomes sufficiently small.

TRAINING OF AUTOENCODER – STEP BY STEP

Example: Movie Recommendation

1 Provide the Input



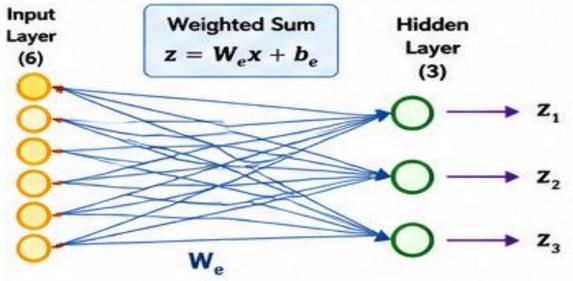
Example Input (x)

$x = [1, 1, 0, 1, 0, 0]^T$

1 = Watched / Liked
0 = Not Watched / Not Liked

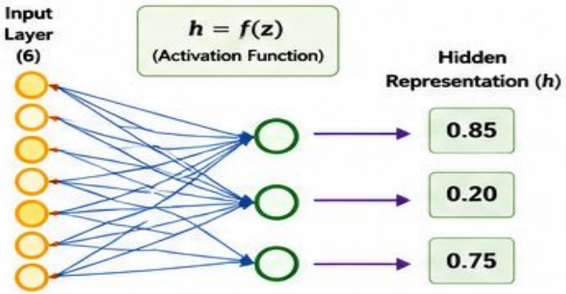
Visible Input Nodes (6)

2 Encoder Processes the Input



Each input is multiplied by weights (W_e) and added with bias (b_e).

3 Hidden Representation is Created (Encoding)

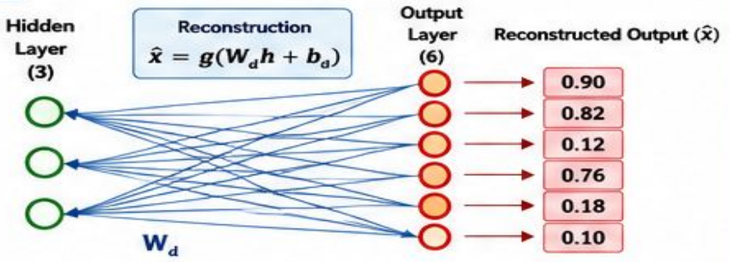


Latent Features (Compressed Representation)

- h_1 : Action interest
- h_2 : Comedy interest
- h_3 : Sci-fi interest

Example: $h = [0.85, 0.20, 0.75]^T$

4 Decoder Reconstructs the Input



The decoder tries to reconstruct the original input from the hidden representation.

5 Compare Output with Original Input

	Original Input (x)		Reconstructed Output ($\hat{x}$)
Movie 1	1		0.90
Movie 2	1		0.82
Movie 3	0	vs.	0.12
Movie 4	1		0.76
Movie 5	0		0.18
Movie 6	0		0.10

The output is close to the original input, but not exact.

6 Calculate Reconstruction Error (Loss)

Movie	x_i	$\hat{x}_i$	$(x_i - \hat{x}_i)^2$
1	1	0.90	0.0100
2	1	0.82	0.0324
3	0	0.12	0.0144
4	1	0.76	0.0576
5	0	0.18	0.0324
6	0	0.10	0.0100

Mean Squared Error (MSE)

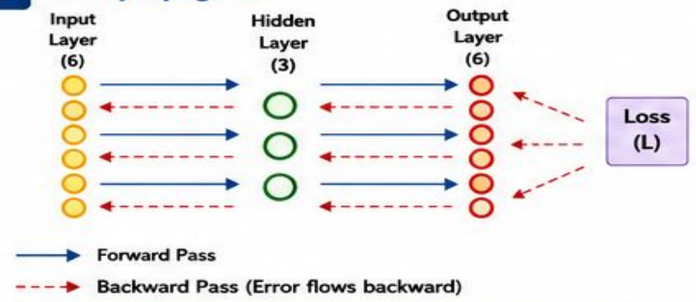
$$L = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

$$= \frac{1}{6}(0.0100 + 0.0324 + 0.0144 + 0.0576 + 0.0324 + 0.0100)$$

$$L = 0.0261$$

Loss shows how much the reconstructed output is different from the original input.

7 Backpropagation



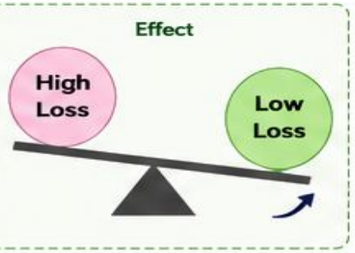
Error (loss) is sent backward to update the weights.

8 Update Weights and Biases

Weight Update (using Gradient Descent)

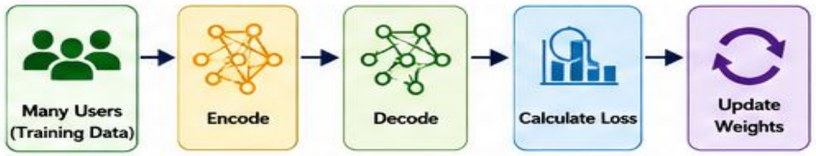
$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

Where,
η : Learning Rate
L : Loss
$\frac{\partial L}{\partial W}$: Gradient of Loss



Weights are adjusted in a direction that reduces the loss.

9 Repeat for All Users (Epochs)



One complete pass through all users = One Epoch

Repeat for many epochs until the loss is very small and the autoencoder learns good representations.

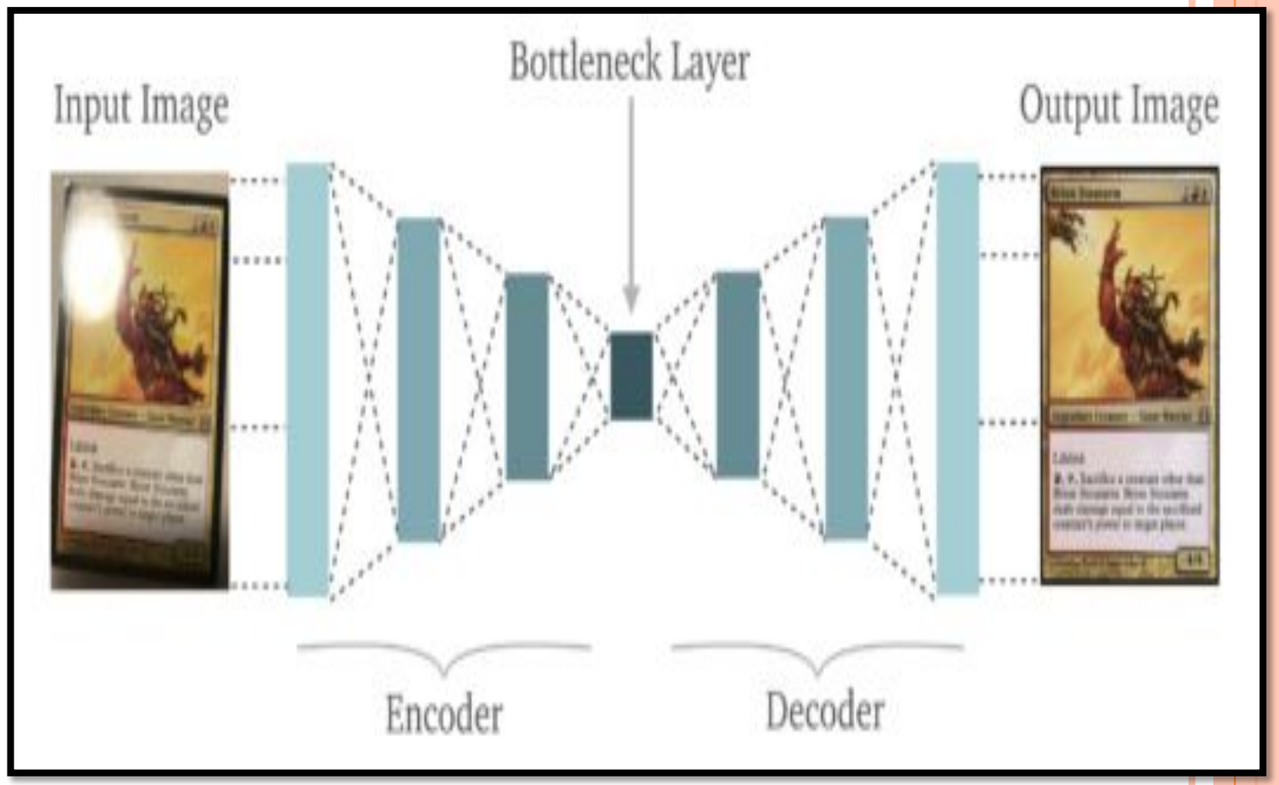
After Training: The hidden layer learns the important patterns (user interests). These patterns can be used to recommend new movies that the user may like.

Applications of Autoencoders

1. Image Coloring: Autoencoders convert **any black and white image into a colored image**. Depending on what is in the **picture** , it is **possible to tell what the colour should be**



2. Feature Variations: Autoencoders **extract required features** from the **original input image** and then **generate noise free output image.**

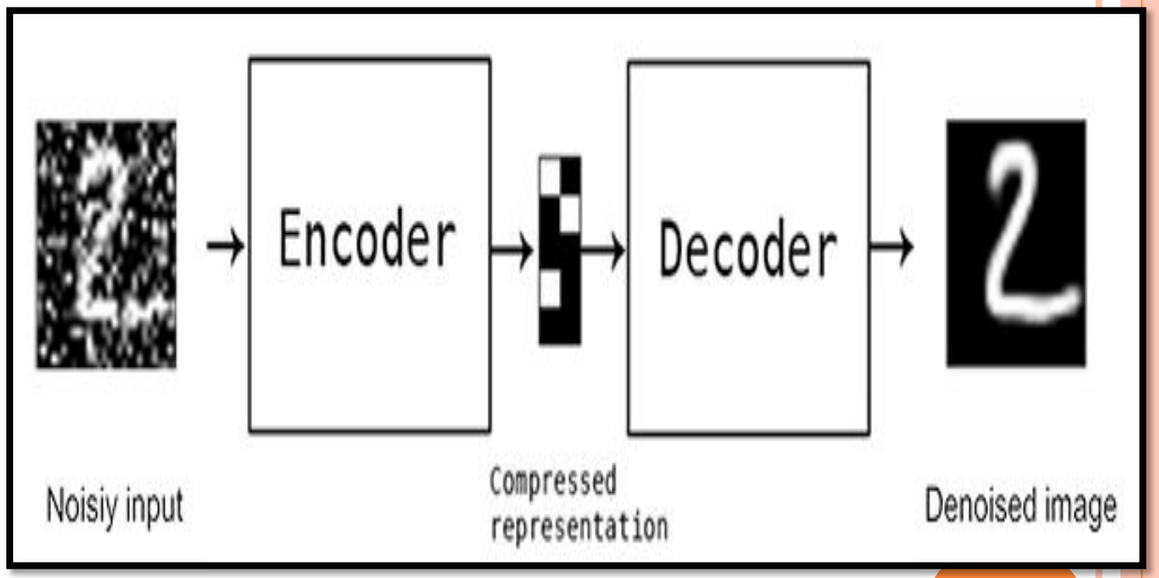


Applications of Autoencoders

3. Dimensionality Reduction: The **reconstructed image** is the **same as our input but reduced dimensions**. It helps in providing the **similar image with a reduced Pixel value.**

4. Denoising Image: A denoising autoencoder is trained to **reproduce the clean image** from the **corrupted/noisy version.**





Applications of Autoencoders

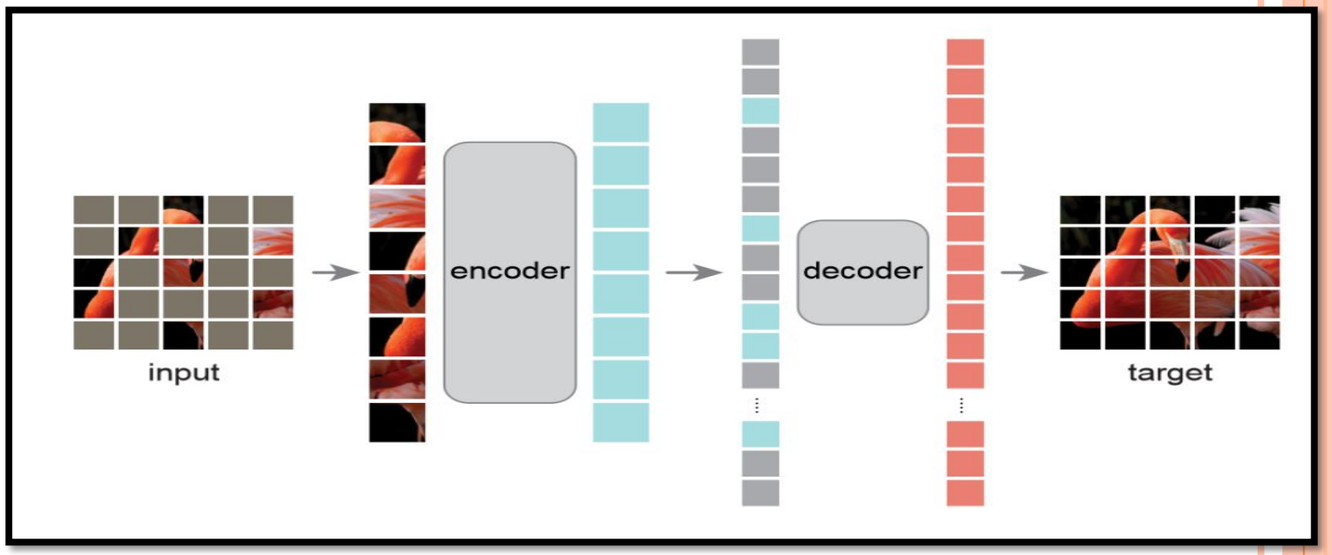
5. Watermark Removal: It is also used for **removing watermarks from images** or to **remove any object** while **filming a video or a movie.**



How Autoencoder Works in Computer Vision

Autoencoder models are commonly used for **image processing tasks** in **computer vision.**

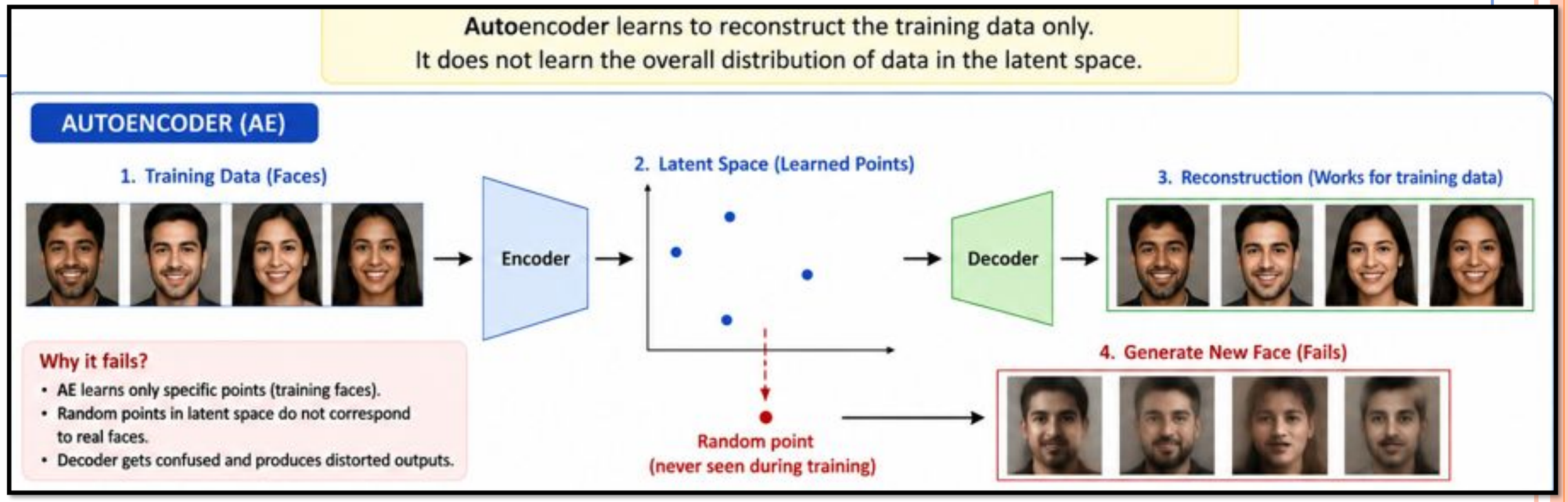
In this **use case**, the **input is an image** and the **output is a reconstructed image**. The model learns **to encode the image into a compressed representation**



Limitations of Auto encoder

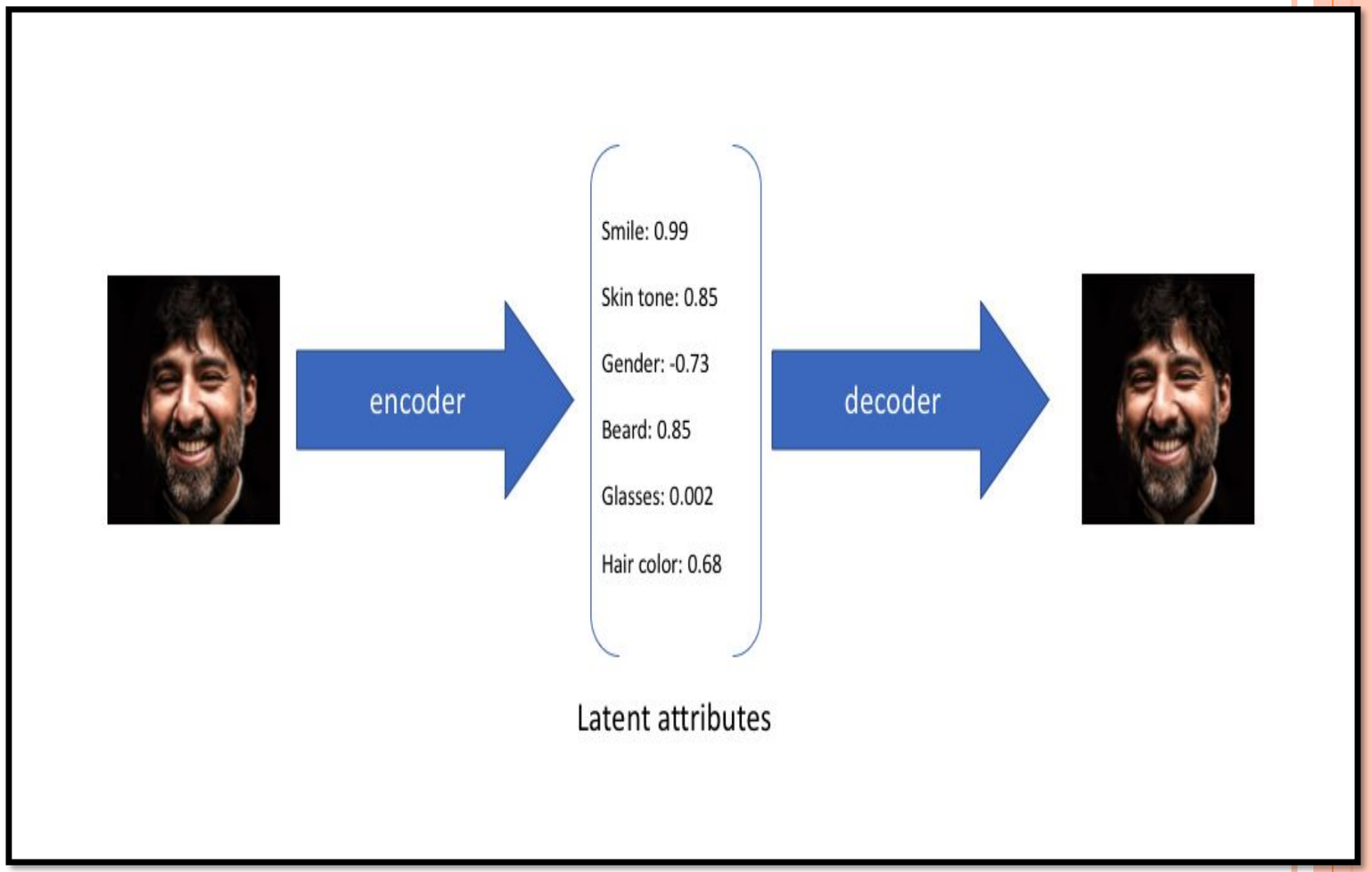
Autoencoders are powerful for **dimensionality reduction, and data compression**, but they have several important limitations.

1. **Cannot Generate New Realistic Data (Basic Autoencoder):** A standard autoencoder **only learns to reconstruct the data it has seen during training**.



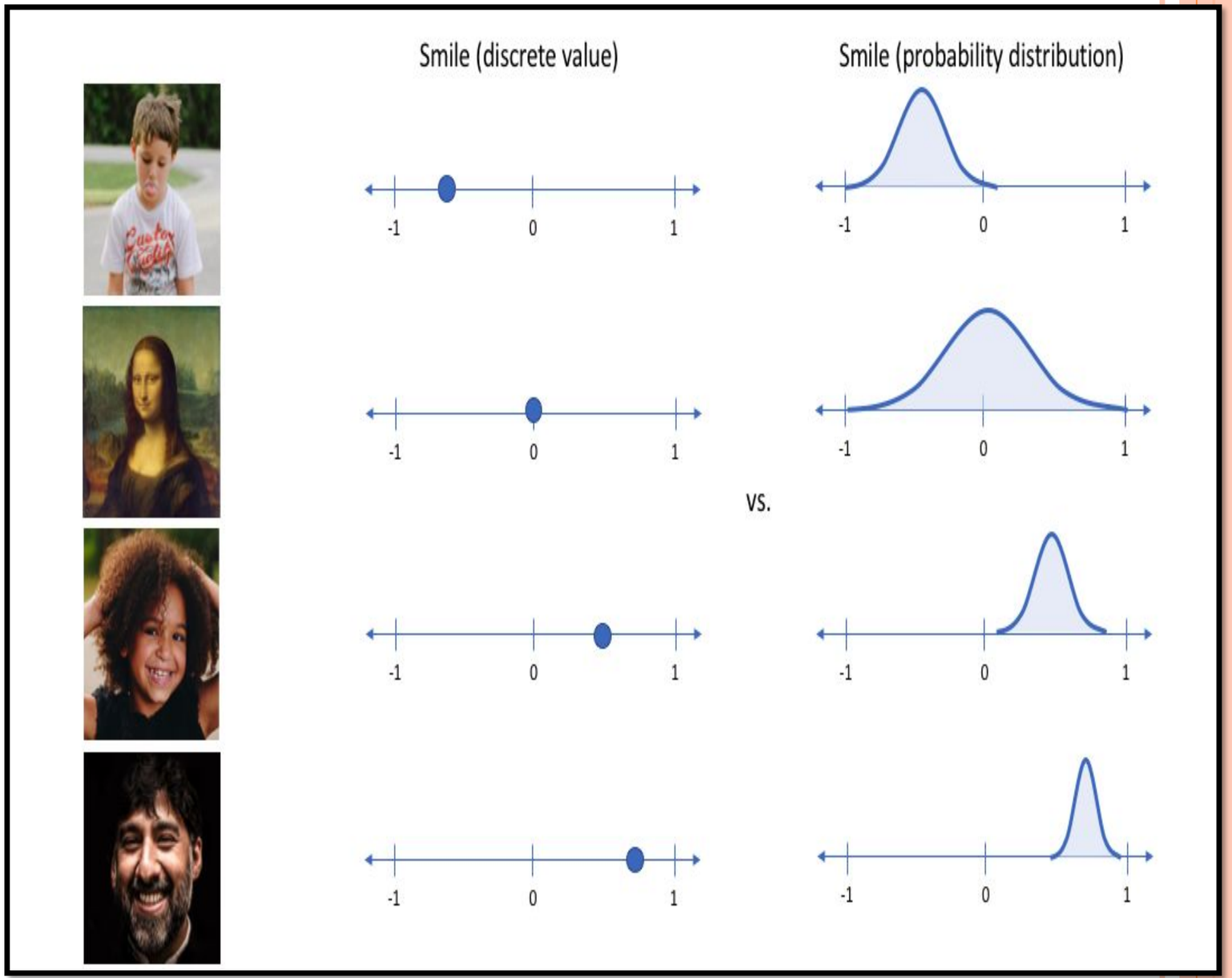
Limitations of Auto encoder

- Let's suppose we've trained an **autoencoder model** on a **large dataset of faces** with **a encoding dimension of 6**.
- An ideal autoencoder will learn **descriptive attributes** of faces **such as skin color, whether or not the person is wearing glasses,** **etc.** in an attempt to describe an observation in **some compressed representation.**



Limitations of Auto encoder

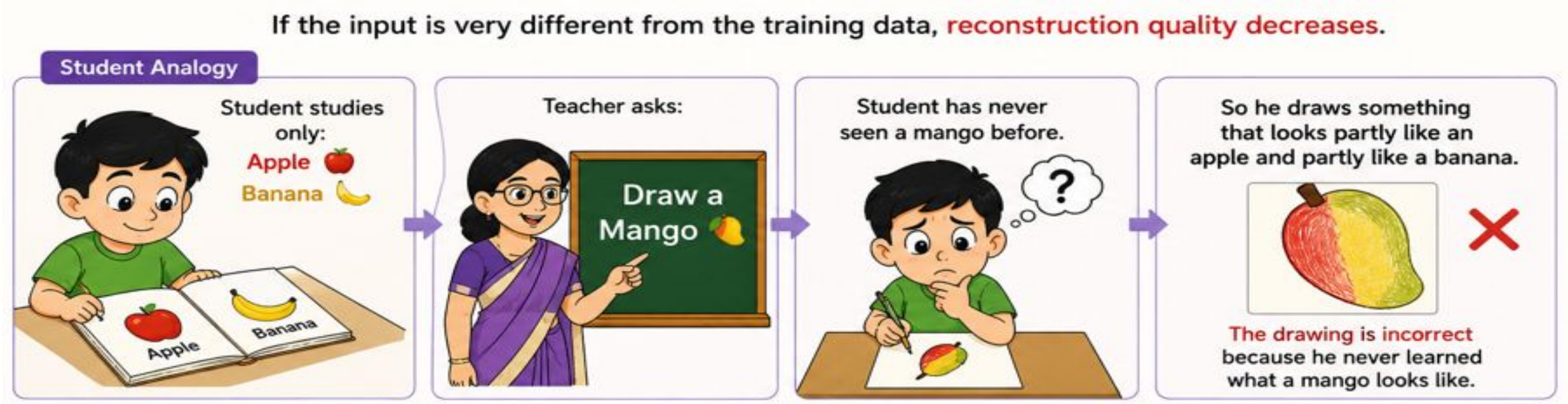
- In **the example above**, we've described the input image in terms of its **latent attributes using a single value** to **describe each attribute**.
- However, we may prefer to represent **each latent attribute** as **a range of possible values**.
- Using a variational autoencoder, we can describe ***latent attributes in probabilistic terms.***



2. Poor Generalization to unseen data: Autoencoders need **many training examples** to learn useful representations.

With limited data:

1. **Overfitting occurs.**
2. **Reconstruction quality decreases.**
3. **Learned features become less meaningful**

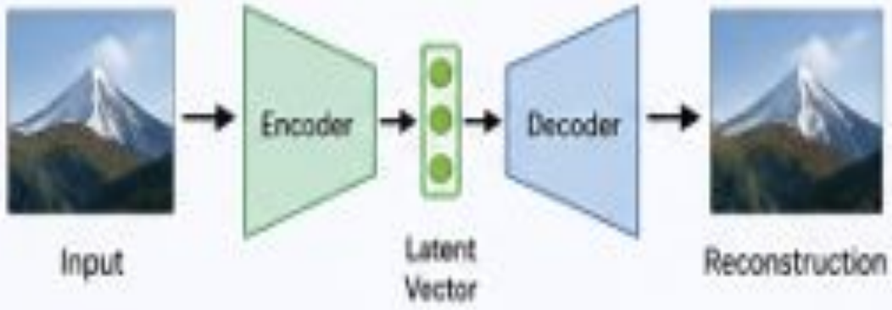


With Large Amount of Data (Good Case)

The autoencoder learns meaningful patterns.



Many Training Examples



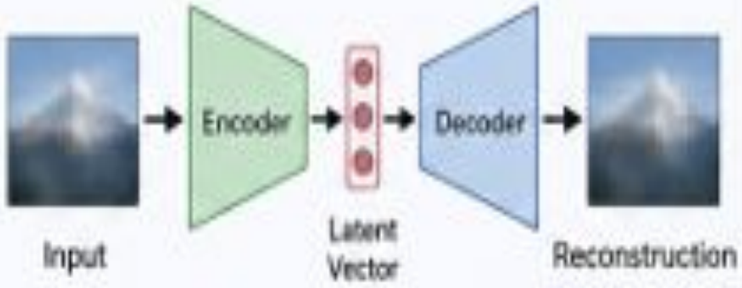
✅ Good reconstruction and learns meaningful features

With Limited Data (Bad Case)

The autoencoder sees too few examples.



Few Training Examples



- ❌ **Overfitting**: Learns training data too well
- ❌ Poor reconstruction on new data
- ❌ Learned features are not meaningful

Effect of Limited Data

Training on Few Examples (Overfitting)



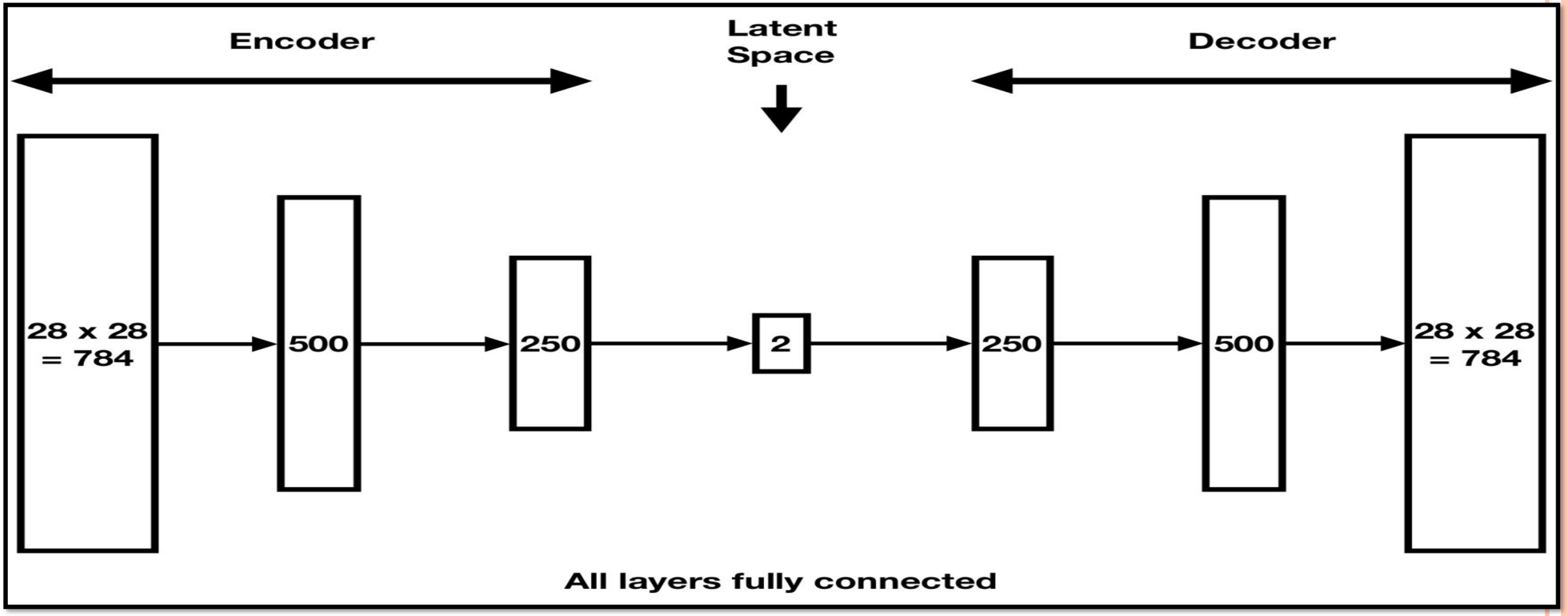
Model memorizes training data, not general patterns.

Reconstruction on New Data (Poor Quality)

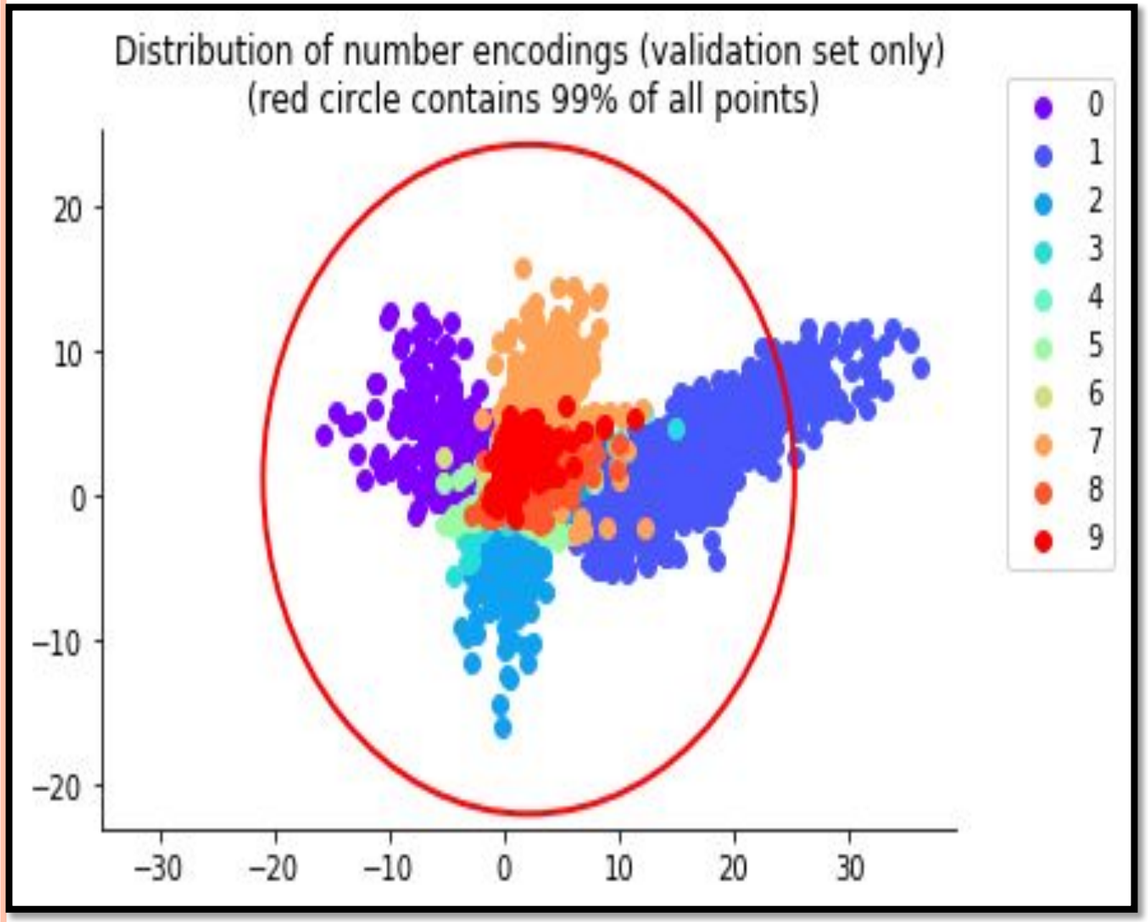


Poor reconstruction and less meaningful features

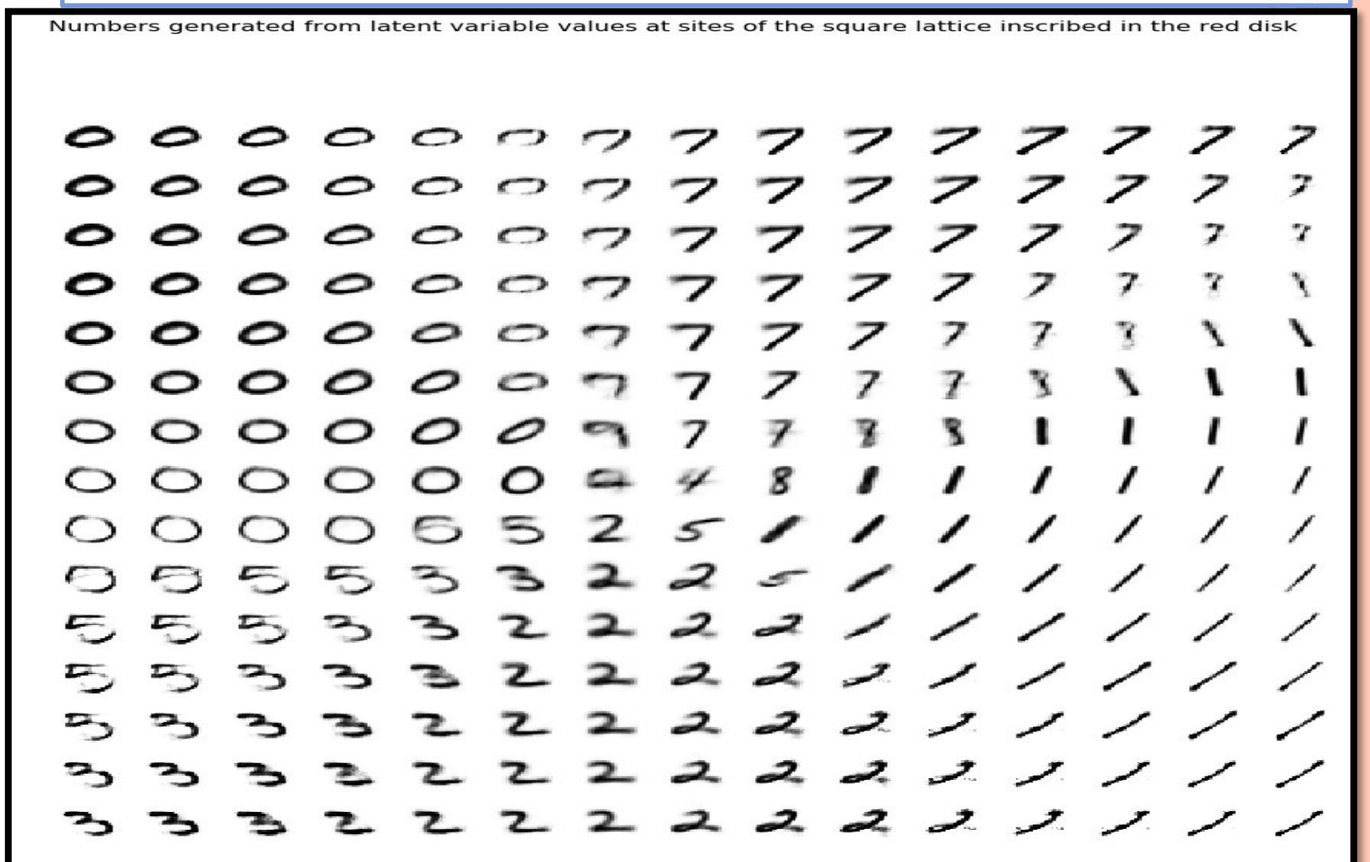
 Another Real Time Example: Using **an autoencoder** that **compresses** the famous **MNIST 28 × 28 handwritten digits dataset**. We use the following **neural network architecture**, which **has a two-dimensional latent space:**



After training our autoencoder (by minimizing its loss function), we find these **encodings of the handwritten digits in our validation set, grouped by their labels 0–9:**



and **pass the vector at each lattice site through the decoder**, then we **generate** the following **"fake" handwritten digits:**



- While **many images** have **good quality,** others are **blurry or incomplete** because **their sites are far from the point cloud.**
- **Drawback is :**
- The **quality of the generated handwritten digits** is **worse than in the two-dimensional case**.
- Because **two-dimensions is usually too small** to **capture the nuances of complicated data**, **this is not good news.**
- **So we can introduced the advanced Topic Variational Autoencoder.**











Learning • Compressing • Reconstructing • Innovating

Thank You!